How Far Does an Edit Reach?

Repairing Derived Facts in Stale KV Caches

Nour Shreif, Hadi Ghazi, Karin Zoabi
Lebanese University

***Abstract*—When a cached document is edited, the cached states of text that depends on the edit become stale, and repairing them means recomputing part of the cache. The published policies recompute a fixed window after the edit or a window at the text that carries the answer; both fail when the answer is a derived fact stated far from the edit, and the contiguous window from the edit to the dependent text has been proposed but not tested. We build a benchmark of derived-fact edits on Qwen3-8B, Llama-3.1-8B and Mistral-7B, varying the distance to the dependent text (0 to about 1,700 tokens), the number of hops (2 or 3) and the number of dependent facts (1 or 3), with checks that the full re-read answers correctly and the stale cache does not. A path window whose stopping point comes from a word index built at edit time recovers the answer margin almost completely in every explicit-dependency cell on all three models (margin recovery 0.99 to 1.00, answer accuracy 95 to 100%) while recomputing 29 to 65% of the tokens that recomputing to the end needs. A fixed window at the same average cost recovers 0.44 to 0.81, the carrier window fails on Llama, falls short on Mistral and fails on every model at three hops, an appended erratum works on Llama but answers correctly only 11 to 54% of the time on the other two, and a dependency graph built by a small LLM before the edit recovers 0.09 to 0.53. Controls show that what repairs the cache is reaching the dependent sentences together with the text that carries the edit to them: random positions at the same budget recover at most 0.36, and the gap without the dependent sentences recovers nothing. When the dependency is paraphrased ("the sixth plan" against "Plan six"), the path window still repairs the cache if it knows where to stop, but the word index cannot find the dependents; a gpt-4.1 dependency graph built before the edit, combined with one asked at edit time, recovers 0.84 to 0.98. A graph asked only at edit time finds the text the edit made relevant and misses the text it made irrelevant, whose stale states still assert the old answer (0.21 against 1.01, depending on which comes first). Positions picked by attention to the edited sentence also succeed at a fixed 512 tokens in every cell, including paraphrased ones, but they need the model's attention statistics and a budget that is not known in advance.**

***Index Terms*—KV cache, document editing, cache repair, retrieval-augmented generation, ripple effects**

I. Introduction

Serving systems keep the key-value (KV) cache of long documents so that later questions do not pay for re-reading them. When a document is edited, every cached state computed after the edited text was computed from the old version, and the cache is stale. Re-reading the whole document restores it but costs a full prefill, so a repair policy recomputes only part of the cache. Mao et al. [1] compare such policies at a fixed budget and find that a contiguous window recomputed right after the edit (EditLocal) beats importance-based selection, but that it fails when the answer is a derived fact stated far from the edit: an edited alias that selects between two unchanged lookup rows 250 or 1,500 tokens later. A window placed at those rows (CarrierWindow, which needs an oracle) works on Qwen and much less on Llama. The authors name a window spanning the path from the edit to the dependent text as the natural next step and leave it untested. A different line appends a correction instead of recomputing [2], tested on direct policy facts but not on derived ones at a distance.

We test the path window, and we ask what any policy has to recompute for a derived fact to be repaired. The practical obstacle is knowing where the dependent text ends. We find it with a deployable rule built at edit time: a word index that starts from the words the edit changed and follows the rare names and codes of the sentences it reaches. For dependencies that share no word with the edit we follow our earlier proof of concept, which combined such an index with an LLM-built dependency graph, and test LLM graphs built before the edit and queried at edit time. Our contributions are:

- a benchmark of derived-fact edits that varies distance, hops, fan-out, distracting mentions of the edited entity and paraphrased dependencies, with every edit length-preserving in three tokenizers and per-model checks that the full re-read answers correctly and the stale cache does not (Section III);
- a comparison of 40 repair policies on Qwen3-8B, Llama-3.1-8B and Mistral-7B, in which the edit-time path window is the only model-free policy that repairs every explicit-dependency cell on every model at less than the cost of recomputing to the end (Section V);
- controls that locate what repair requires: the dependent sentences and the text that carries the edit to them, rather than a budget large enough (Section VII);
- for paraphrased dependencies, evidence that a strong LLM can find the dependents, that a complete dependency set must include the text the edit made irrelevant as well as the text it made relevant, and that combining a graph built before the edit with one queried at edit time covers both (Section VI);
- evidence that the erratum is strongly model-dependent, and that positions picked by stored attention, as in PatchKV, can repair at long range if their budget is large enough, while they fail at the 32-token budget Mao et al. tested (Sections V and VII).

II. Related Work

Repairing stale caches. Mao et al. [1] define the in-place repair problem, the margin-recovery metric we use, the derived alias-to-lookup benchmark we extend, and the EditLocal and CarrierWindow policies; their code is not released, so we reimplement both. PatchKV [3] repairs dynamically edited contexts with an edit-local window plus distant blocks chosen by stored attention, on multi-hop QA workloads without a derived-fact construction. Chunk-reuse methods such as CacheBlend [4] recompute selected tokens when independently cached chunks are joined, a related but different staleness. Li [2] argues that the cache stores the model's notes about the text, and that appending an erratum is a cheap, robust way to change a decision; its tests use gated policy facts on one model, and its two-hop refreshes of the edited field alone recover almost nothing. Cai and Cai [5] discuss when appended corrections are preferable to edits in sparse event caches.

Ripple effects. Knowledge-editing work calls the downstream consequences of an edited fact its ripple effects [6] and tests them with multi-hop questions [7]. Our cells play the same role for a cached document instead of model weights.

III. Benchmark

Items. Each document is real WikiText-103 text [8] (validation paragraphs with none of the benchmark's names or code-like tokens) with a few inserted sentences. About 600 tokens of text precede the edited sentence, so recomputing to the end is cheaper than a full re-read, and about 600 tokens follow the last dependent sentence. Nine cells of 50 items each:

- *direct*: the edited sentence states the code itself ("Company Varnex's code is Q7" becomes "R6"), a check that refreshing the edit is enough;
- *derived*, $d \in \{0, 250, 1500\}$: Mao et al.'s construction. "Company Varnex uses alias Delta" becomes "Lambda", and two unchanged rows ("Alias Delta maps to code Q7", "Alias Lambda maps to code R6") follow at distance d, in random order;
- *3 hops*: alias rows map to groups and group rows map to codes, each pair about 250 tokens further;
- *fan-out 3*: three attribute pairs (code, tag, room) about 250 tokens apart; the question asks for one;
- *noisy*: the derived cell at $d = 250$ with four extra sentences about the same company (office, staff, founding year, sponsorship) in the gap and the tail;
- *indirect*, $d \in \{250, 1500\}$: the derived construction with no shared word, taken from our proof of concept. "Company Varnex uses the third plan" becomes "the sixth plan", and the rows read "Plan three maps to code Q7" and "Plan six maps to code R6", in random order.

The question gives the rule ("Company Varnex's code is the code of the alias it uses") and asks for the code. Measured distances from the end of the edit to the first dependent sentence have medians of 317 to 362 tokens in the $d = 250$ cells and 1,500 to 1,740 in the $d = 1500$ cell; documents run from about 1,450 to 3,550 tokens.

Validity. Every edit changes a contiguous run of tokens inside the edited sentence and leaves the token count unchanged in all three tokenizers, so cached positions stay aligned; items that fail this in any tokenizer are redrawn. Each model then keeps an item only if the full re-read improves the new-minus-old margin over the stale cache by at least one nat per token and prefers the new answer, the full re-read generates the new code, and the stale cache does not. Qwen3 keeps 322 of the 350 explicit-dependency items, Llama 263 and Mistral 260. The rejects are items with too small a margin gain (12, 40 and 26 for Qwen3, Llama and Mistral), items whose full re-read did not generate the new code (12, 21 and 42), and items whose stale cache already generated it (4, 26 and 22). The indirect items are harder: even after a full re-read, the models often cannot resolve "the sixth plan" to "Plan six", and they keep 61, 25 and 32 of 100.

IV. Policies and Setup

Policies. Every policy starts from the stale cache and recomputes a set of positions of the edited document with the same operator; we report the tokens it recomputes.

- *Floors and ceilings*: no repair; the edited tokens only; recompute from the edit to the end (*suffix*); a full re-read (*full*).
- *Fixed windows*: EditLocal with $K \in \{32, 64, 128, 256, 512, 1024\}$ tokens after the edit, and a window to the end of the edited paragraph.
- *Oracle carriers*: Mao's CarrierWindow (32 tokens from the first row the question needs), and the dependent sentences alone (*rows*).
- *Path windows*: recompute from the edit to the end of the furthest dependent sentence, found by the construction's ground truth (oracle), by the *edit-time index*, by an LLM graph, or by both.
- *Erratum* [2]: append "[STATE UPDATE] Company Varnex: alias $\rightarrow$ Lambda (was Delta). This overrides any earlier value and conclusion." after the document, with the cache left stale, with the edited tokens refreshed, or with EditLocal@32.
- *Attention*: rank downstream positions by the attention they paid to the edited sentence during the original prefill (every fourth layer, averaged over heads), and recompute the top positions or 32-token blocks, at the path window's budget or at a fixed 32, 128 or 512; or stop where that attention falls below 25% or 10% of its maximum.
- *Controls*: random positions or random 32-token blocks at the path window's budget, and the gap from the edit up to the first dependent sentence without that sentence (*StopShort*).

Edit-time index. The index needs no model. It takes the words that differ between the old and new edited sentence (here the two aliases), marks every later sentence containing one of them, then adds the names and codes (capitalised words or words containing a digit) that occur in at most three sentences of the document and follows them for up to three hops. The path window recomputes up to the end of the

furthest sentence reached. The *LLM graph* instead asks gpt-4o-mini, once per document and before any edit, for the sentence-level dependency edges of the original text, and follows them from the edited sentence; it cost US$0.18 for all 350 explicit-dependency documents. On the indirect cells we build this graph with gpt-4o-mini and with gpt-4.1, and we also ask each model *at edit time*, given the old and new edited sentence, which later sentences depend on it; the path window then stops at the furthest sentence any chosen source reaches. These graphs cost US$1.55 for the 100 indirect documents.

Models and runtime. Qwen3-8B [9], Llama-3.1-8B-Instruct [10] (the ungated mirror of the same weights) and Mistral-7B-Instruct-v0.3 [11], in bfloat16 on one L40S GPU each. The document sits in the user turn of each model's chat template, with the question after it. Repair uses our own forward pass over a preallocated cache, so that any set of positions can be recomputed in place, each attending to every cached key at or before it. Before each run it is checked against the library: the last-token logits differ by at most 0.8% of their range with identical top-5 tokens, and recomputing every position of a stale cache reproduces a fresh prefill exactly.

Metrics. Margin recovery (MR) [1] is $(m - m_{\text{stale}})/(m_{\text{full}} - m_{\text{stale}})$, where m is the length-averaged log-probability of the new answer minus that of the old one: 0 is no better than the stale cache and 1 is as good as a full re-read. Accuracy is the share of items whose greedy answer (up to 8 tokens) contains the new code and not the old. We report means with item-level bootstrap 95% intervals [12]. A second run of seven reference policies on a fresh pod reproduced every item's MR exactly.

V. Results

The path window repairs every explicit-dependency cell. With the edit-time index, the path window reaches MR 0.99 to 1.00 and 95 to 100% accuracy in all eighteen derived cells (Tables I and II). The index found every dependent sentence and no other sentence on every item, so the oracle path window gives identical numbers. On the direct cell, where the index stops at the edit itself, refreshing the two edited tokens already recovers 0.91 to 0.96.

Fixed windows cannot know how far to go. EditLocal@32 repairs the adjacent rows ($d = 0$, MR 1.00) and nothing beyond them (MR at most 0.06 in every other derived cell), which reproduces Mao et al.'s 0.01 to 0.09 at $d = 250$. Larger fixed windows do not close the gap at equal cost (Fig. 1): a fixed window spends its budget on near items and falls short of far ones, so at the path window's mean cost a fixed window recovers 0.44 to 0.81 (log-interpolated between the two neighbouring budgets of the EditLocal sweep, where that cost lies inside the sweep), and even 1,024 tokens recover nothing at $d = 1500$.

It costs a fraction of the alternatives that also work. Recomputing to the end also recovers 1.00, and so does a full re-read; the path window recomputes 29 to 65% of the suffix's tokens and 18 to 50% of the full re-read's in the distant cells (Table IV). The saving is the text after the last dependent sentence, so it shrinks when dependents sit near the end of the document.

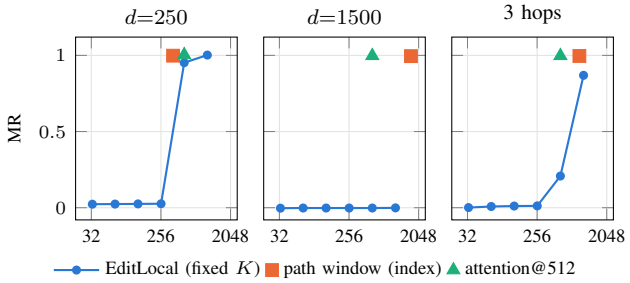


Fig. 1. Margin recovery against tokens recomputed (log scale), mean over the three models. A fixed window recovers only once K passes the dependent text, which differs per item; the path window stops at the right place on each item.

The carrier window depends on the model and fails at three hops. Recomputing 32 tokens at the rows the question needs recovers 0.92 to 0.95 on Qwen3 at every distance but 0.30 to 0.46 on Llama and 0.56 to 0.69 on Mistral, as Mao et al. found for Qwen and Llama. With three hops it recovers nothing on any model; the row the question needs is the second stage, and the group rows it builds on stay stale.

The erratum is cheap but unreliable. Appending a 26 to 34-token correction to a cache whose edited tokens are refreshed gives 68 to 100% accuracy on Llama, but 29 to 54% on Qwen3 and 11 to 43% on Mistral, and it is weakest at three hops and fan-out. MR exceeds 1 on Llama because the erratum adds evidence a re-read of the edited document does not contain.

A graph built before the edit misses dependencies. The LLM graph recovers 0.84 to 0.85 when the rows are adjacent but only 0.09 to 0.53 elsewhere. Its edges reach 76% of the dependent sentences when they are adjacent, 53% at $d = 250$, 27% at $d = 1500$, 25% at three hops, 15% at fan-out and 9% in the noisy cell. It also favours the rows of the old alias, which the original document's logic uses, over those of the new alias, which the edit makes relevant (48% against 24% at three hops). The edit-time index reaches every dependent sentence because it starts from the words the edit changed.

VI. Paraphrased Dependencies

The repair still works; finding the dependents is the problem. When the stopping point is known, the path window recovers 0.98 to 1.02 on the indirect cells as well (Table III). The word index finds no dependent sentence (its recall is 0.00 to 0.02), so it stops early or at a filler sentence that happens to contain "third" or "sixth", and recovers 0.11 to 0.42. gpt-4o-mini fails as well, before the edit (recall 0.02 to 0.09) and at edit time (0.00 to 0.02). gpt-4.1 finds most dependents: its graph built before the edit reaches 66 to 75% of them and recovers 0.73 to 0.94, and asked at edit time it reaches 54 to 57% and recovers 0.38 to 0.77. Combining the two recovers 0.84 to 0.98, with 80 to 97% correct answers. Attention@512 recovers 0.98 to 1.03 here too, without any graph.

TABLE I

MARGIN RECOVERY (MEAN; 1 = AS GOOD AS A FULL RE-READ) ON THE DERIVED CELLS. BOLD: THE PATH WINDOW WITH THE EDIT-TIME INDEX. EL32: EDITLOCAL@32; CARRIER: MAO'S CARRIERWINDOW (ORACLE); ROWS: THE DEPENDENT SENTENCES ONLY (ORACLE); ERRATUM: EDITED TOKENS PLUS APPENDED CORRECTION; LLM GRAPH: PATH WINDOW FROM A GRAPH BUILT BEFORE THE EDIT; RANDOM: RANDOM POSITIONS AT THE PATH WINDOW'S BUDGET; STOPSHORT: THE GAP WITHOUT THE DEPENDENT SENTENCES; ATTN: POSITIONS RANKED BY ATTENTION TO THE EDITED SENTENCE, FIXED BUDGET.

Model	Cell	el32	carrier	rows	erratum	LLM graph	random	StopShort	attn@128	attn@512	path (index)
Qwen3-8B	derived, d=0	1.00	0.94	0.94	0.43	0.85	0.00	0.00	1.00	1.00	**1.00**
	derived, d=250	0.01	0.92	0.92	0.49	0.52	0.05	0.02	0.96	1.00	**1.00**
	derived, d=1500	−0.02	0.93	0.93	0.55	0.26	0.36	−0.02	0.60	1.00	**1.00**
	3 hops	0.02	0.01	0.85	0.24	0.18	0.14	0.03	0.40	1.00	**1.00**
	fan-out 3	0.00	0.50	0.92	0.30	0.19	0.26	0.00	0.56	0.98	**1.00**
	noisy, d=250	0.00	0.95	0.94	0.49	0.09	0.07	0.01	0.85	1.00	**1.00**
Llama-3.1-8B	derived, d=0	1.00	0.46	0.46	0.92	0.84	−0.02	0.00	1.00	1.00	**1.00**
	derived, d=250	0.04	0.30	0.30	1.09	0.53	0.07	0.04	0.92	1.00	**1.00**
	derived, d=1500	0.02	0.37	0.37	1.19	0.30	0.32	0.03	0.65	0.99	**0.99**
	3 hops	0.06	0.04	0.27	0.89	0.29	0.12	0.05	0.48	1.00	**1.00**
	fan-out 3	0.03	0.21	0.32	0.76	0.31	0.17	0.03	0.68	1.00	**1.00**
	noisy, d=250	0.01	0.32	0.32	0.97	0.11	0.06	0.01	0.80	1.00	**0.99**
Mistral-7B	derived, d=0	1.00	0.69	0.69	0.48	0.84	0.00	0.00	1.00	1.00	**1.00**
	derived, d=250	0.02	0.62	0.62	0.48	0.42	0.01	0.02	0.85	1.00	**0.99**
	derived, d=1500	−0.01	0.56	0.56	0.44	0.25	0.29	−0.01	0.62	1.00	**1.00**
	3 hops	−0.07	−0.02	0.56	0.15	0.29	0.00	−0.05	0.35	0.99	**0.99**
	fan-out 3	0.03	0.33	0.66	0.30	0.31	0.21	0.03	0.40	1.00	**1.00**
	noisy, d=250	0.02	0.68	0.68	0.49	0.11	0.07	0.02	0.68	1.00	**1.00**

TABLE II

ANSWER ACCURACY (%, GREEDY ANSWER CONTAINS THE NEW CODE AND NOT THE OLD) FOR THE SAME POLICIES AND CELLS AS TABLE I.

Model	Cell	el32	carrier	rows	erratum	LLM graph	random	StopShort	attn@128	attn@512	path (index)
Qwen3-8B	derived, d=0	100	100	100	50	84	0	0	100	100	**100**
	derived, d=250	0	98	98	54	50	2	0	96	100	**100**
	derived, d=1500	0	94	94	38	27	31	0	48	100	**98**
	3 hops	3	3	92	31	11	11	3	33	100	**100**
	fan-out 3	2	52	98	29	21	19	2	60	98	**100**
	noisy, d=250	0	100	100	48	9	9	0	91	100	**100**
Llama-3.1-8B	derived, d=0	100	31	29	92	86	0	0	100	100	**100**
	derived, d=250	6	19	16	94	55	10	6	77	97	**100**
	derived, d=1500	4	36	32	100	32	18	0	54	100	**100**
	3 hops	9	3	31	84	19	6	6	47	100	**100**
	fan-out 3	3	13	29	68	32	6	3	61	100	**100**
	noisy, d=250	0	36	36	100	7	5	0	64	100	**98**
Mistral-7B	derived, d=0	100	69	69	43	84	0	0	100	100	**100**
	derived, d=250	5	66	66	26	45	5	5	82	97	**97**
	derived, d=1500	26	67	67	21	36	44	26	59	100	**100**
	3 hops	14	18	55	23	41	23	14	23	100	**95**
	fan-out 3	17	39	72	11	44	22	22	33	100	**100**
	noisy, d=250	2	55	55	34	11	2	2	70	98	**100**

A dependency set must include what the edit made irrelevant. The edit-time query is reliable when the old plan's row comes first and fails when the new plan's row comes first (Table V: 1.01 against 0.21). Asked about the edited sentence, the model returns the row the edit made relevant and leaves out the row of the old plan. When that row comes later, the path window stops before it, and its stale states, computed while the old plan was in force, still treat its code as the company's code. A graph built before the edit links the edited sentence to the old plan's row, so the two sources see complementary halves, and their combination is close to the oracle whichever row comes first (0.86 and 1.01). The swapped documents of our proof of concept showed the mirror image for explicit dependencies: a graph built before the edit misses the row the edit makes relevant.

VII. WHAT REPAIR REQUIRES

Location, not budget. Spending the path window's exact budget on random positions recovers 0.00 to 0.36, and on random 32-token blocks at most 0.56. The budget alone does not repair the cache.

The dependent sentences are necessary. StopShort, which recomputes the whole gap but stops before the first dependent sentence, recovers at most 0.05 in every distant cell on every model.

TABLE III

INDIRECT CELLS ("THE SIXTH PLAN" AGAINST "PLAN SIX"): MARGIN RECOVERY OF THE PATH WINDOW BY HOW THE DEPENDENTS ARE FOUND, AGAINST ATTENTION@512 AND THE ORACLE. n: ITEMS KEPT. BEFORE: GRAPH BUILT FROM THE ORIGINAL DOCUMENT; AT EDIT: THE MODEL ASKED, GIVEN THE OLD AND NEW EDITED SENTENCE, WHICH LATER SENTENCES DEPEND ON IT; BOTH: THE PATH WINDOW STOPS AT THE FURTHEST SENTENCE EITHER REACHES. ACCURACY IS FOR "BOTH".

Model	Cell	n	word index	gpt-4o-mini before	gpt-4.1 before	gpt-4.1 at edit	gpt-4.1 both	attn@512	oracle	acc. (%)
Qwen3-8B	d=250	35	0.32	0.06	0.88	0.55	**0.97**	1.00	1.00	97
	d=1500	26	0.42	0.01	0.75	0.61	**0.88**	0.99	1.01	88
Llama-3.1-8B	d=250	15	0.11	0.05	0.86	0.38	**0.93**	1.02	0.99	93
	d=1500	10	0.39	0.31	0.73	0.71	**0.84**	0.99	1.01	80
Mistral-7B	d=250	13	0.24	−0.11	0.94	0.57	**0.98**	1.03	1.02	85
	d=1500	19	0.42	0.12	0.91	0.77	**0.93**	0.98	0.98	89

TABLE IV

MEAN TOKENS RECOMPUTED (MEAN OVER MODELS): THE PATH WINDOW, RECOMPUTE TO THE END, AND A FULL RE-READ, WITH THE PATH WINDOW'S SHARE OF THE FULL RE-READ.

Cell	path	suffix	full	path/full
derived, d=0	20	791	1526	1%
derived, d=250	368	1265	2008	18%
derived, d=1500	1628	2514	3257	50%
3 hops	905	1782	2514	36%
fan-out 3	1512	2391	3142	48%
noisy, d=250	387	1308	2058	19%

TABLE V

INDIRECT CELLS POOLED OVER MODELS AND DISTANCES, SPLIT BY WHICH OF THE TWO ROWS COMES FIRST: MARGIN RECOVERY OF THE GPT-4.1 PATH WINDOWS (61 ITEM-MODEL PAIRS WITH THE NEW PLAN'S ROW FIRST, 57 WITH THE OLD PLAN'S).

Dependents found by	new row first	old row first
gpt-4.1, before the edit	0.84	0.86
gpt-4.1, at edit time	0.21	1.01
gpt-4.1, both	0.86	1.01
oracle	0.99	1.01

On two models they are not sufficient. Recomputing the dependent sentences alone recovers 0.85 to 0.94 on Qwen3 but 0.27 to 0.46 on Llama and 0.56 to 0.69 on Mistral. On those models the text between the edit and the rows also carries the old alias forward, and repairing it is part of the repair. This is the mechanism behind the carrier window's model dependence.

Attention can find the places, not the budget. Positions ranked by the attention they paid to the edited sentence recover 0.99 to 1.00 at the path window's budget in every distant cell, and 0.98 to 1.00 at a fixed 512 tokens, which is cheaper than the path window when the dependents are far (512 against about 1,600 tokens at $d = 1500$ and fan-out 3). At 32 tokens they recover at most 0.15 at a distance and at 128 tokens 0.35 to 0.96, so the budget they need is not known in advance; the two stopping rules we fixed before running were unreliable (25%: at most 0.14 at a distance; 10%: -0.03 to 1.00). Attention needs no notion of shared words, and at 512 tokens it also repairs the paraphrased cells (0.98 to 1.03). It does need the attention that downstream positions paid to the edited span, which a deployed system would have to store at prefill time or pay for at edit time; we did not charge that cost. These results agree with both earlier papers rather than contradicting Mao et al.'s "contiguity, not importance": at their 32-token budget, attention-ranked positions fail here too, and the success at 512 tokens matches PatchKV's use of stored attention to choose distant blocks [3].

VIII. DISCUSSION

A derived fact is repaired when the recomputed set contains the sentences that use the edited value and, on some models, the text that carries it to them. A fixed budget cannot guarantee that, because how far an edit reaches is a property of the document. For explicit dependencies, the edit-time index answers that question without touching the model: it costs one pass over the document's words, needs no prefill statistics and no LLM call at edit time, and gives each item its own stopping point. For paraphrased dependencies it fails, and a strong LLM is needed; a small one is not enough on documents of this length. Attention selection is the strongest model-internal alternative and the cheaper one at long range, provided its statistics are available and its budget happens to be large enough; combining the index's stopping point with attention-ranked positions inside it is a natural next policy. A dependency set has two halves: the text the edit makes relevant and the text it makes irrelevant, whose stale states still assert the old conclusion. A graph of the original text sees the second half and cannot contain links the edit creates; a question asked at edit time about the new sentence sees the first half and misses the second. Any practical builder therefore has to look at both versions of the edited sentence, as the word index does by construction and as the combination of the two LLM graphs does. The edit-time LLM query has a price the word index does not: an API call takes seconds, while in our runtime the path window's repair takes about 50 ms and a full re-read about 200 to 225 ms (medians). It is practical as a background step or as a fallback when the word index finds no dependent sentence, not on the critical path of every edit. Finally, the erratum's success on one model and failure on two others suggests that whether a model trusts a late correction over earlier text is a model property, which a policy cannot assume.

IX. Limitations

Most dependencies are explicit: in seven of nine cells every dependent sentence repeats the alias or group name, which suits a word index. Its perfect precision on those cells is built in: the background text was filtered so that no benchmark name or code appears in it, so the index could not make a false hit; on the indirect cells, whose filler was not filtered for ordinals, it reached 2.1 and 3.8 unrelated sentences per document on average at the two distances. Every dependency is an inserted template sentence in real background text; we test no real documents or real edits. The paraphrased cells cover one kind of rewording, and the models keep few of their items (25 and 32 of 100 for Llama and Mistral), so their intervals are wide. The LLM graphs ran once each, at temperature 0. Edits are single and length-preserving. Documents are at most about 3,550 tokens and models are 7 to 8B parameters. The attention policies' cost of obtaining attention statistics is not counted. Llama-3.1 weights came from an ungated mirror. Kept items differ by model, and accuracy is judged on at most 8 generated tokens. Every model call ran once in bfloat16; the reference re-run agreed exactly.

X. Conclusion

Repairing a stale KV cache after an edit to a derived fact requires reaching the text the edit affects, and the reach differs from document to document. A path window that stops where an edit-time word index says the dependent text ends repairs every explicit-dependency cell of our benchmark on three models at 18 to 50% of the cost of a re-read, where fixed windows, carrier windows, appended errata and small-LLM graphs each fail somewhere. When the dependency is paraphrased, the same window works once a strong LLM finds the dependents, and the dependents must include the text the edit made irrelevant as well as the text it made relevant. Attention-selected positions can do the same, more cheaply at long range, when the model's attention statistics and a large enough budget are available.

Reproducibility

The benchmark builder, the repair runtime with its integrity checks, all results and the analysis are released. The study used about two and a half hours of three L40S GPUs (about US$7.60) and US$1.73 of API calls.

References

[1] M. Mao, W. Mackey, and X. Lin, "Contiguity, not importance: Budgeted repair of stale KV caches after document edits," 2026, arXiv:2609.17983.

[2] B. Li, "Models take notes at prefill: KV cache can be editable and composable," 2026, arXiv:2606.17107.

[3] G. Yang, R. Guo, S. He, S. Chen, Y. Hu, and K. Li, "PatchKV: Efficient KV cache recovery for dynamically edited LLM contexts," 2026, arXiv:2609.26219.

[4] J. Yao *et al.*, "CacheBlend: Fast large language model serving for RAG with cached knowledge fusion," in *Proc. European Conference on Computer Systems (EuroSys)*, 2025, arXiv:2405.16444.

[5] Z. Cai and Z. Cai, "Compute globally, materialize locally: The memory contract of sparse event-KV," 2026, arXiv:2607.23693.

[6] R. Cohen, E. Biran, O. Yoran, A. Globerson, and M. Geva, "Evaluating the ripple effects of knowledge editing in language models," *Transactions of the Association for Computational Linguistics*, vol. 12, 2024.

[7] Z. Zhong, Z. Wu, C. D. Manning, C. Potts, and D. Chen, "MQuAKE: Assessing knowledge editing in language models via multi-hop questions," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[8] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," in *Proc. International Conference on Learning Representations (ICLR)*, 2017.

[9] Qwen Team, "Qwen3 technical report," 2025, arXiv:2505.09388.

[10] A. Dubey *et al.*, "The Llama 3 herd of models," 2024, arXiv:2407.21783.

[11] A. Q. Jiang *et al.*, "Mistral 7B," 2023, arXiv:2310.06825.

[12] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.